SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA0515** **Slot B**

COURSE OUTCOME COVERED
CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 10%

QUESTIONS:4 **Total Marks 20** **Duration :60 Minutes**

Annexure A

Mock Interview

Code of Conduct:

I M. Pranay (192525382)_certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Pranay

Rubrics for Evaluation

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	***Marks(100)***
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

ANSWERS

1. Complete OpenCV Pipeline for Multiple Images

Answer:

A complete OpenCV pipeline for processing multiple images from a folder is:

Image Folder → Read Images → Preprocess → Process → Display/Save → Release Resources

Steps

1. **Import OpenCV**
   - Use import cv2.
   - Use os or glob to obtain image file names.
2. **Read images**
   - Get all image paths from the folder.
   - Read each image using cv2.imread().
3. **Check image validity**
   - If cv2.imread() returns None, skip the invalid image.
4. **Preprocessing**
   - Resize images using cv2.resize().
   - Convert to grayscale using cv2.cvtColor().
   - Apply Gaussian blur using cv2.GaussianBlur() when noise reduction is required.
5. **Processing**
   - Perform the required operation such as edge detection using cv2.Canny().
6. **Display**
   - Display the processed image using cv2.imshow().
7. **Save**
   - Save the processed image using cv2.imwrite().
8. **Optimization**
   - Process images sequentially instead of loading all images into memory.
   - Resize large images before processing.
   - Skip corrupted or unsupported files.

Challenges

- Large image sizes increase memory usage.
- Invalid or corrupted images may fail to load.
- Different image formats may require handling.
- Processing many images can take more time.

Conclusion:

Using a sequential folder-processing pipeline with image validation, resizing, preprocessing, and efficient memory management provides reliable and efficient OpenCV image processing.

2. Video File Not Loading in OpenCV

Answer:

If a video does not load properly, I would follow a systematic debugging approach.

Step 1: Check the Video Path

Use:

cv2.VideoCapture("video.mp4")

Ensure that the file path and file name are correct.

Step 2: Check Whether Video Opened

```
cap = cv2.VideoCapture("video.mp4")

IF cap.isOpened() == False
   Video cannot be opened
END IF
```

If isOpened() returns false, check the file path, file format, and codec.

Step 3: Check Frame Reading

Use:

ret, frame = cap.read()

- ret = True → frame successfully read.
- ret = False → frame could not be read.

Step 4: Check Video Format and Codec

- Verify that the video format is supported.
- Try a commonly supported format such as MP4.
- Check whether the required codec is available.

Step 5: Check File Integrity

Try opening the video with another media player. If it cannot play there either, the video may be corrupted.

Step 6: Check Backend/Environment

If the video works elsewhere but not in OpenCV:

- Check the OpenCV installation.
- Check available video backends/codecs.
- Test with another video file.

Step 7: Release Resources

After processing:

```
cap.release()
cv2.destroyAllWindows()
```

Conclusion:

The debugging process should check **file path → isOpened() → frame reading → format/codec → file integrity → OpenCV environment**. This systematically identifies the source of the problem.

3. Overlay Text and Shapes on Live Video

Answer:

The pipeline is:

Camera → Capture Frame → Draw Shapes/Text → Display → Repeat

Implementation Steps

1. Open the camera using:

cv2.VideoCapture(0)

2. Read each frame using:

ret, frame = cap.read()

3. Draw a rectangle using:

cv2.rectangle()

4. Draw a circle using:

cv2.circle()

5. Draw a line using:

cv2.line()

6. Add text using:

cv2.putText()

7. Display the result using:

cv2.imshow()

8. Continue processing until the user presses q.

Pseudocode

```
BEGIN

Open camera

WHILE camera is available

  Read frame

  IF frame is not available
    BREAK
  END IF

  Draw rectangle
  Draw circle
  Draw line
  Add text

  Display frame

  IF user presses 'q'
    BREAK
  END IF

END WHILE

Release camera
Close windows

END
```

Efficiency

- Keep the camera resolution appropriate for real-time processing.
- Avoid unnecessary image copies.
- Draw directly on the current frame.
- Process only the required regions when possible.

Conclusion:

OpenCV drawing functions such as **rectangle(), circle(), line(), and putText()** can efficiently overlay information on a live video stream.

4. Image Appears Too Dark When Read

Answer:

If an image appears too dark, the problem may be caused by low illumination, incorrect image processing, or improper brightness/contrast.

Proposed Pipeline

Read Image → Check Image → Brightness/Contrast Enhancement → Histogram Equalization → Display

Step 1: Read the Image

image = cv2.imread("image.jpg")

First check whether the image was successfully loaded.

Step 2: Convert to Grayscale if Required

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

Step 3: Improve Brightness and Contrast

Adjust brightness and contrast using:

enhanced = alpha * image + beta

- alpha controls contrast.
- beta controls brightness.

Step 4: Histogram Equalization

For a grayscale image:

equalized = cv2.equalizeHist(gray)

This redistributes intensity values and can improve visibility.

Step 5: Adaptive Enhancement

For uneven lighting, techniques such as **CLAHE (Contrast Limited Adaptive Histogram Equalization)** can improve local contrast.

Step 6: Display

Display the enhanced image using:

cv2.imshow()

Why This Improves Visibility

- Increasing brightness makes dark regions more visible.
- Contrast enhancement separates objects from the background.
- Histogram equalization improves the distribution of intensity values.
- CLAHE is useful when different parts of the image have different illumination.

Conclusion:

A dark image can be improved by applying **brightness/contrast adjustment, histogram equalization, or CLAHE**, depending on the lighting problem. This produces a clearer image while preserving important visual information.